642
602

⚠ この記事は最終更新日から1年以上が経過しています。

G*Advent Calendar(Groovy,Grails,Gradle,Spock...) Advent Calendar 2014 | 15日目

@opengl-8080

2014年12月15日に投稿　2019年01月13日に更新

Groovyを知らない人のためのbuild.gradle読み書き入門

Java, Groovy, gradle

Adobe Creative Cloud
コンプリートプランが今だけ
3,960(税込)円/月
10月13日(水)まで
今すぐ購入
Adobe

アジャイル開発を実践！
ヤフーの
バックエンドエンジニア
積極採用中
YAHOO! JAPAN

はじめに
イコール代入
スペース区切り
コロン区切り
ドット区切り
波括弧で階層化
　クロージャ内で参照しているプロパティやメソッドはどこからやってきた？
　クロージャが暗黙的に持つ delegate 変数
　スクリプトブロックの DSL ドキュメントを参照する
　スクリプトブロックとドット区切りを使用した記法を相互に書き換える
メソッド呼び出しのあとの波括弧ブロック
未定義のプロパティやメソッドはどこからやってきた？
　便利メソッドの探し場所としての Project クラス
　プラグインが追加するプロパティ・メソッド
タスク定義
【おまけ】タスク名に未定義の変数？が使用されている点について
参考

これは、 G* Advent Calendarの15日目の記事です。
昨日は @int128 さんの Gradle Slashプラグインをリリースしました #gadvent でした。
明日は @tyama さんです。

はじめに

Gradle 便利ですよね。便利すぎて Ant や Maven には戻れないです。

なにが良いって、設定ファイルである `build.gradle` の記述量が、 Ant の `build.xml` や Maven の `pom.xml` と比べると非常に少なくて済むのが良いです。

`build.gradle` は、設定ファイルと言いつつも、その実体は Groovy で書かれたスクリプトファイルです。

Gradle は、 Groovy の持つメタプログラミング機能や省略記法などを利用して、設定を簡潔に記述できるようになっています。

これはメリットなのですが、一方で Groovy のことを知らないと、「なんでこんな記述で設定できるんだろう？」とか「同じ設定なのに記法がいろいろある？！」などの混乱を招くデメリットにもなります。

例えば、以下のような `build.gradle` があったとします。

```
apply plugin: 'application'
apply plugin: 'eclipse'

mainClassName = 'sample.Main'

repositories {
    mavenCentral()
}

dependencies {
    compile 'org.apache.commons:commons-lang3:3.3.2'
    testCompile('junit:junit:4.11') {
        transitive = false
    }
}

startScripts.applicationName = 'sample'

eclipse {
    project {
        name = 'SampleProject'
    }
}
```

簡単な Java プロジェクトの `build.gradle` です。

これだけでも、「イコール代入」、「スペース区切り」、「コロン区切り」、「ドット区切り」、「波括弧で階層化」、「メソッド呼び出しのあとの波括弧ブロック」などなど、様々な記法を見ることができます。

はじめの頃なら、なんとなく意味が分かればそれで十分です。しかし、ある程度使いこなしてくるとオプションのカスタマイズが必要になります。そのとき、これらの記法がどういう意味を持つのかを理解できていないと、簡単な設定の変更にも苦労することになります。

Groovy は省略記法やシンタックスシュガーが多くある言語です。このため、同じ実装を様々な方法で記述することができます。

つまり、 `build.gradle` を自由にカスタマイズできるようになるためには、Groovy の省略記法を理解しておく必要があるということです。

この記事では、これら `build.gradle` の記述がどういう仕組みになっているのかについてまとめたいと思います。

イコール代入

まずは、もっとも単純な記法です。

```
mainClassName = 'sample.Main'
```

Java など多くの言語で使われているものと同じで、 `=` は代入演算子です。

つまり、上記は `mainClassName` というプロパティに `'sample.Main'` という文字列を設定しています。

これは、特に疑問もなく読むことができると思います。

スペース区切り

```
compile 'org.apache.commons:commons-lang3:3.3.2'
```

プロジェクトが依存するライブラリとして、 Apache の Commons Lang を定義している部分です。

`compile` と `'org.apache.commons:commons-lang3:3.3.2'` の間には `=` がなく、スペースしかありません。

この設定は、実は以下のように記述した場合と同じになります。

```
compile('org.apache.commons:commons-lang3:3.3.2')
```

`compile` はプロパティの代入ではなく、メソッドの呼び出しを行っています。なので間に `=` を挟むことはありません。

なぜこのようにできるのかというと、 Groovy には**「メソッドの呼び出しは丸括弧を省略することができる」**というルールがあるためです。

ただし、引数が１つも存在しない場合は省略できません。

上述の `build.gradle` でいうと、 `mavenCentral()` がちょうど引数無しのメソッドなので、丸括弧を省略することなく記述しています。

コロン区切り

```
apply plugin: 'application'
```

今度は、コロン `:` です。

`apply` はメソッドの呼び出しで、丸括弧が省略されています。

括弧を省略せずに書くと以下のようになります。

```
apply(plugin: 'application')
```

Java しかやったことのない人には馴染みがないかもしれませんが、これはいわゆる**名前付き引数**というものです。

名前付き引数では、名前と値を１セットにして引数を渡します。名前付き引数には、引数を宣言的に指定できる・引数の順番を覚える必要がない、などのメリットがあります。

Groovy の名前付き引数はシンタックスシュガーで実現されています。

上記の設定を Java しかしらない人でも分かるように置き換えると、以下のようになります。

```
Map map = new HashMap()
map.put('plugin', 'application')

apply(map)
```

Groovy では、メソッド引数が `Map` を受け取る場合、 `key : value` のようにコロン区切りで `Map` のエントリーを渡すことができます。

複数のエントリーを渡したい場合は、以下のようにカンマ区切りで記述します。

```
myMethod hoge: 'Hoge', fuga: 'Fuga', piyo: 'Piyo'
```

`apply plugin: 'application'` という設定は、メソッドの丸括弧の省略と名前付き引数の記法が組み合わさることで実現されています。

ドット区切り

```
startScripts.applicationName = 'sample'
```

これ自体は、そんなに難しい記法ではないと思います。

`startScripts` というオブジェクトが持つ `applicationName` というプロパティに対して、 `'sample'` という文字列を設定しています。

波括弧で階層化

```
eclipse {
    project {
        name = 'SampleProject'
    }
}
```

上記設定は、「 `eclipse` プロパティの `project` プロパティがもつ `name` プロパティに `'SampleProject'` という値を設定する」という意味になります。

波括弧で囲まれた部分は**クロージャ**といいます。

クロージャは、任意のコードを記述できるブロック要素で、コード上の任意の場所で生成したり変数に代入したり実行したりすることができます。

```
def closure = { println 'Hello Closure!!' } // クロージャを定義して変数に代入

closure() // クロージャを実行
```

```
> gradle
Hello Closure!!
```

最初の階層化された記法は、省略記法をやめてクロージャを変数に入れるようにすると、以下のようになります。

```
def projectClosure = { name = 'SampleProject' }
def eclipseClosure = { project(projectClosure) }

eclipse(eclipseClosure)
```

`eclipse` と `project` はプロパティを参照していたのではなく、実はメソッドを呼び出していたのです。
そして、波括弧の正体はクロージャで、 `eclipse()` と `project()` の引数として渡されていたのでした。

クロージャ内で参照しているプロパティやメソッドはどこからやってきた？

省略記法をやめた実装をよくよく見てみると、 `name` や `project()` はどこにも宣言されていないのに突然現れています。

試しにそれぞれのクロージャを単独で動かすと、次のようにエラーが発生します。

```
def eclipseClosure = { project(projectClosure) }
eclipseClosure() // eclipseClosure を単独で実行
```

```
> gradle
FAILURE: Build failed with an exception.

（中略）

> Could not find method project() for arguments [build_4f9ulb02pvaod4a1l53oeu1k1l$_run_
```

`Could not find method project()` つまり、 `project()` というメソッドは存在しない、というエラーです。

確かに `project()` というメソッドは定義されていませんが、このクロージャを `eclipse()` メソッドに渡して実行すると、エラーは起こらなくなります。

ここが Groovy の力が遺憾なく発揮されている部分の１つです。

この動作の実現には、 delegate という仕組みが利用されています。

ここを理解すると、 `build.gradle` がよりいっそう読みやすく、書きやすくなります。

ここを理解すると、 `build.gradle` がよりいっそう読みやすく、書きやすくなります。

クロージャが暗黙的に持つ delegate 変数

クロージャの中では、暗黙的に使用できる変数がいくつかあります。

その中の１つに、 `delegate` という変数があります。

クロージャ内では、この `delegate` 変数に設定されているオブジェクトのプロパティやメソッドが暗黙的に参照できるようになります。

```
// クロージャを宣言
def closure = {
    println message
    method()
}

// クロージャの delegate に Hoge クラスのインスタンスを設定
closure.delegate = new Hoge()

// クロージャを実行
closure()

def class Hoge {
    def message = 'hoge'

    def method() {
        println 'Hoge.method()'
    }
}
```

上記の実装を `build.grdle` の任意の場所に記述して Gradle を実行すると、以下のように出力されます。

```
hoge
Hoge.method()
```

クロージャの `delegate` プロパティに `Hoge` クラスのインスタンスを設定することで、クロージャ内では定義されていなかった `message` プロパティや `method()` メソッドにアクセスできるようになっています。

ここで、 `eclipse` の設定に戻って、クロージャ内の `delegate` 変数を出力してみます。

```
eclipse {
    println 'eclipse.delegate.class = ' + delegate.class
    project {
        println 'project.delegate.class = ' + delegate.class
        name = 'SampleProject'
    }
}
```

```
> gradle
eclipse.delegate.class = class org.gradle.plugins.ide.eclipse.model.EclipseModel_Decor
project.delegate.class = class org.gradle.plugins.ide.eclipse.model.EclipseProject_Dec
```

それぞれのクロージャ内で、 `delegate` オブジェクトにインスタンスが設定されているのがわかります。

Gradle では、この `delegate` の仕組みによって、波括弧（クロージャ）を使った簡潔な設定が可能になっています。

この仕組を理解していると、その波括弧内（クロージャ内）で参照できるプロパティやメソッドは何なのかを意識することができ、 DSL のドキュメントも読みやすくなるのではないかと思います。

スクリプトブロックの DSL ドキュメントを参照する

クロージャを引数に受け取るメソッドは**スクリプトブロック**と呼び、 DSL ドキュメント上も `Script Block` という名前で説明されています。

例えば `eclipse()` メソッドの説明は これ です。

`Delegates to` のところに書かれているリンクが、クロージャの `delegate` に設定されるインスタンスのクラスです。

つまり、そのスクリプトブロック内で参照できるプロパティやメソッドは、 `Delegates to` に記載されているリンクを開けば確認できるということです。

スクリプトブロックとドット区切りを使用した記法を相互に書き換える

前述のドット区切りを使用した `startScripts` の設定は、スクリプトブロックを使用した記法に書き換えることができます。

```
startScripts {
    applicationName = 'sample'
}
```

また、スクリプトブロックを使用していた `eclipse` の設定は、ドットを使用した記法に切り替えられます。

```
eclipse.project.name = 'SampleProject'

// 他にもこんなことも可能
eclipse {
    project.name = 'SampleProject'
}
```

一方はプロパティの参照なのに対して、もう一方はメソッドの呼び出しで、それぞれは本質的には別物です。

にもかかわらず、それぞれの記法は相互に変換ができています。

この相互変換ができる理由は単純で、それぞれの API が提供されているからです。

- eclipse プロパティ
- eclipse スクリプトブロック

基本的に Gradle のプラグインは、プロパティとスクリプトブロックの両方を提供しているので、ドット区切りとスクリプトブロックは相互に変換できるようになっています。

このおかげで、設定項目が少ない場合はドット区切りを使い、多くなる場合はスクリプトブロックを使ってまとめて設定するという使い分けができるようになっています。

メソッド呼び出しのあとの波括弧ブロック

```
testCompile('junit:junit:4.11') {
    transitive = false
}
```

`testCompile()` というメソッドが呼ばれた後に、波括弧のブロックが現れています。

この波括弧も、クロージャです。

この設定は、以下のように書いたものと同じになります。

```
testCompile('junit:junit:4.11', { transitive = false })
```

つまり、クロージャは `testCompile()` メソッド引数の１つです。

Groovy には、メソッド引数の最後がクロージャの場合、クロージャの記述をメソッド呼び出しの丸括弧の外に記述できるというルールがあります。

未定義のプロパティやメソッドはどこからやってきた？

ここまで、 `apply()` や `dependencies()` 、 `mainClassName` といったプロパティやメソッドは、宣言されていないにも関わらず普通に使うことができていました。

これらの暗黙的に使用できるプロパティやメソッドは、 Project というクラスに定義されています。

`build.gradle` では、クロージャにおける `delegate` と同じように、 `Project` が持つプロパティやメソッドを暗黙的に使用することができるようになっています。

```
apply plugin: 'application'

mainClassName = 'sample.Main'

println mainClassName
println this.project.mainClassName
```

```
> gradle
sample.Main
sample.Main
```

`Project` のインスタンスは `this.project` に格納されており、暗黙的に参照したプロパティやメソッドは、この `this.project` のプロパティおよびメソッドとして実行されるようになっています。

便利メソッドの探し場所としての Project クラス

`Project` は暗黙的に参照できるので、 `build.gradle` で使用できる便利メソッドなどは、だいたいこのクラスに定義されています。

例えば、 `files()` や `copy()` といったよく見かけるメソッドは、この `Project` クラスに定義されています。

なので、「何か便利メソッドないかなぁ」と思ったときは `Project` クラスの DSL ドキュメントを見てみると、掘り出し物を見つけたりできるかもしれません。

プラグインが追加するプロパティ・メソッド

Java しか知らないと不思議な感じがするかもしれませんが、 Groovy ではプロパティやメソッドを実行時にクラスに追加することができます。

`apply plugin: '****'` でプラグインを読み込むと、プラグインは前述の `this.project` インスタンスに、プロパティやメソッドを動的に追加します。

`eclipse` などのプロパティは、プラグインが追加することで暗黙的に使用できるようになっています。

プラグインを追加するとどんなプロパティやメソッドが追加されるかは、各プラグインの説明ページが、Project の DSL ドキュメントに記載されています。

タスク定義

最後はタスクの定義です。

```
task myTask << {
    println 'my task'
}
```

これを、省略記法やシンタックスシュガーなしで記述すると、次のようになります。

```
task(myTask).leftShift({ println 'myTask' })
```

`task()` は `Project` クラスに定義されたタスクを登録するためのメソッドです。

そして、 `<<` は Task インターフェースの `leftShift()` というメソッドのシンタックスシュガーで、引数で指定したクロージャをタスクの最後に実行するように登録します。

なぜ `<<` という演算子で `leftShift()` メソッドが実行されるのかというと、Groovy には、既定の名前で定義されたメソッドの呼び出しを簡単な演算子に置き換えられる仕組みがあるためです。これを**演算子オーバーロード**と言います。

`leftShift()` という名前で定義されたメソッドは、 `<<` 演算子で実行できるようになっているのです。

また、タスクの定義には `doLast()` というメソッドもあります。このメソッドも、引数で渡したクロージャをタスクの最後に実行するよう登録します。なので、どちらのメソッドでもタスクが定義できるようになっています。

【おまけ】タスク名に未定義の変数？が使用されている点について

前述の設定では、 `myTask` は未定義の状態で使用されています。

なので、そのままなら「そのようなプロパティは存在しません」という意味のエラーが発生するはずですが、実際はそうなりません。

なぜこのようなことができるのか、残念ながら仕組みは私もよく分かっていません。

Groovy では、未定義のプロパティが参照された場合に任意の処理を挟んで、好きな値を返すように実装することができます。

```
this.metaClass.propertyMissing = { name ->
    System.out.println('name = ' + name)
    return 'property missing'
}

println 'this.unknownProperty = ' + this.unknownProperty
```

```
name = unknownProperty
this.unknownProperty = property missing
```

しかし、この方法だと `task()` メソッドの呼び出しのとき以外に未定義のプロパティを参照しても `propertyMissing()` が実行されてしまい、エラーが発生しなくなります。

Gradle のソースコードを落として Eclipse のデバッガを使って確かめてみましたが、 `task()` メソッドが呼ばれたときに引数には `"myTask"` という文字列が渡されていました。

ところが、どのタイミングで `myTask` が `"myTask"` という文字列に変換されているのかは分かりませんでした。。。

以上です。

Gradle をはじめた当初、私はここに書いているようなことを知りませんでした。

そのため、なぜこれらの記述で設定ができるのかがわからず、 `build.gradle` に対してなんだか取っ付きにくいイメージがありました。

しかし、ここで書いていることがわかってくると、実はそれほど複雑ではないことがわかり、取っ付きにくいイメージはだんだんと薄れていきました。

Gradle をさわり始めたばかりで、同じようなイメージを抱いている方の一助となれば幸いです。

さらに次のステップに進む場合は以下。

Gradle のタスク定義のあれこれ - Qiita

参考

- Gradle User Guide
- Gradle DSL Version 2.2-20140924021627+0000

@opengl-8080

ただのSE。Java好き。

フォロー

TIS株式会社

創業50年超のSIerです。

https://www.tis.co.jp/

HITACHI
Inspire the Next
システム設計から障害対応まで。
日立のクラウドビジネス推進センタに迫る
Qiita Zine

最上流工程を目指す
IT・WEBエンジニアの
転職
年収700万円～
社内SE・PL経験者歓迎
マイナビ IT AGENT
転職相談はこちら ▶

関連記事 Recommended by LOGLY

Gradle のタスク定義のあれこれ
by opengl-8080

Gradle を完全に理解した人が、何も分からなくなるための第一歩
by opengl-8080

【Vue】一体いつから括弧無しメソッドと括弧有りメソッドが同じだと錯覚していた？
by _masa_u

Swift5.1のattribute全解説｜全27種
by shtnkgm

クラウド型AI-OCRの技術者が語る、研究開発寄りのプロダクト開発のやりがい
PR 日立

水俣病を止められなかった「企業城下町」の構造
PR 國學院大學

この記事は以下の記事からリンクされています

Spring Bootでサーバーサイドkotlin入門 からリンク 10 months ago
Re:ゼロから始めるbuild.gradle.kts生活 からリンク 1 year ago
Gradle を完全に理解した人が、何も分からなくなるための第一歩 からリンク 1 year ago
Androidプロジェクトのビルドシステムを理解するのに役立ったサイト一覧 からリンク 2 years ago
MacでGroovyを動かす からリンク 2 years ago
過去の10件を表示する

コメント

@nakawai 2017-07-31 18:21

> 前述の設定では、 myTask は未定義の状態で使用されています。

私も気になって調べてみたのですが、以下の回答が参考になるっぽいです。
https://stackoverflow.com/a/27584555

LGTM 2

@opengl-8080 2017-07-31 21:57

おぉ、情報ありがとうございます。

たぶん標準の文法を拡張するようなことはしてるんだろうなぁとは思ってましたが、標準で構文木を書き換える仕組みがあるんですねー

LGTM 0

投稿する

編集 プレビュー

テキストを入力

画像を選択 0B / 100MB 投稿

記事投稿イベント開催中

QiitaAzure
記事投稿キャンペーン
Microsoft Azure

Azure Kubernetes Serviceに関する記事を投稿しよう！
2021/10/11~2021/11/10
イベント詳細を見る

すべて見る

© 2011-2021 Increments Inc.